Comprehensive Analysis of Activation Functions in Handwritten Digit Recognition

Koushik Dgs[1*] and Sayan Sikder[1]

[1]School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India .

Contributing authors: sayan.sikder@vit.ac.in;

Abstract

Mathematical transformations represent the way information is transformed within each neuron thereby having a substantial impact on the overall performance of any model the process of transforming the information determines how signals are transmitted through several layers and the efficiency with which separable patterns can occur if the mathematical transformations do not induce non-linear behaviour then the neural network is restricted in its ability to effectively distinguish between complex visual representations necessary for identifying handwritten symbols thus non-linear response mechanisms are employed in practical designs of neural networks there are different types of non-linear response functions in machine learning such as the sigmoid relu leaky and mish the work described in this paper explores a training technique where the non-linear response mechanism adapts over time as the model is trained initially a simple non-linear transformation is utilised to facilitate rapid extraction of basic patterns conversely a smooth non-linear transformation is used later in training to foster greater detail in feature extraction and provide consistency in updating parameters experiments performed using the mnist handwritten digit database demonstrate that the adaptive strategy of changing the form of the non-linear response mechanism results in performance comparable to or greater than that obtained when using a single form of the non-linear response function.

Keywords: Activation functions, Neural networks, Sigmoid, ReLU, Leaky ReLU, Mish, MNIST dataset, time-slicing.

1 Introduction

Artificial intelligence ai as of late has evolved dramatically providing paths for the computer to process information like the human mind could in the past an exciting advancement came from being able to have a system discern hand-writtenhand-written numbers [29] many believed identifying hand written numbers would always remain a human task property examples can be found in postal sorters digitizing paper documents and detecting license plates from vehicles [1] currently the mnist data set is world-renowned as the ideal data set for developing and testing sound neural network-based algorithms for recognizinghand-written numbers [29, 30] the initial challenges faced when developing a neural network for recognizing hand-written digits are the different ways in which each individual writes ie style of writing the pattern of writing and the direction of the stroke the difference in angles when writing and degrading effects of the environment a model describing the complex relationships between the individual digit images presented to the neural network as input and their labels must be constructed in a flexible manner suitable for recognizing hand-written digits neural networks understand such relationships because they are based on training with an extensive amount of data [31]

In line with previous studies the current study presents a method for training deep networks that utilizes both relu and mish as complementary activation functions in a single training process relu was used during the early stage to allow the network to quickly learn its basic feature set and reach an initial convergence point [2–4] after this point the activation function was switched from relu to mish because of the improved ability of mish to produce smoother gradient transitions and better predictability [8, 14] relu was used for the first five epochs and mish for all epochs thereafter to provide fast early learning and greater accuracy and stability for later optimization [20, 24] to evaluate this scheduling the relu-mish hybrid network was tested against networks trained on the mnist dataset with individual activation functions including sigmoid relu and leaky relu under the same conditions [10, 29, 31, 32] the relu-mish hybrid network demonstrated similar or superior performance because of its ability to achieve rapid initial convergence followed by smooth gradient-based refinement during subsequent epochs.

Key Contributions of This Study

- Investigation of activation function influence on MNIST handwritten digit recognition.
- Comparative evaluation of Sigmoid, ReLU, Leaky ReLU, and Mish under identical architecture and training conditions.
- Proposal of a Hybrid ReLU → Mish activation scheduling strategy.
- Two-phase training design:
  - Early epochs: ReLU for fast convergence.
  - Later epochs: Mish for smooth gradient refinement and stability.
- Empirical validation showing hybrid activation achieves competitive or superior performance compared to single-function models.

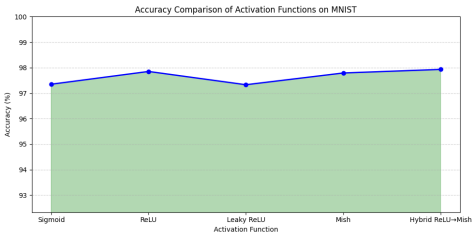


Fig. 1

- Demonstration that activation functions need not remain static during training.

2 LITERATURE SURVEY

The training characteristics of a DNN or deep neural network are essentially determined by the mathematical rules that specify how signals can be transferred through each of the individual neurons. The activation functions that exist within a DNN control the rate at which a model learns and the accuracy of a model when presented with inputs that it hasn't seen before. Therefore, the selection of a proper activation function is one of the most important factors involved in both the efficient operation of a DNN and its ability to generalize correctly, as discussed by Goodfellow et al. [1]

The original neural networks utilized primarily sigmoid activation functions or "S-shaped" response curves which provided outputs that could be interpreted as probabilities. It soon became apparent, however, as the networks grew deeper and more complex that these types of functions restricted the flow of error information throughout the training process due to the effects of saturation during back-propagation, as explained by LeCun et al. [2]. Therefore, adjusting the weights of the neurons in the network was problematic and, as a result, it was very difficult to develop accurate models for visual recognition problems (such as classifying handwritten digits) because of the small differences between two sets of input (two distinct digit patterns) would not be present [1].

To address these difficulties with training, researchers developed a number of alternative forms of activation that do not saturate. One such activation function that became common usage is the Rectified Linear Activation Function (ReLU), which is both efficient and has been proven to accelerate learning rates considerably in very deep architectures (DNN's), as demonstrated by Nair and Hinton et al. [3]. However, the ReLU will, in some cases, cause certain neurons to become inactive permanently, as discussed by Maas et al. [4]. As such, there has been a decorative introduction of alternative activations to the ReLU, which allow for small, negative values to be produced by those neurons which would otherwise not have been able to provide any output. One example of this would be the Leaky ReLU, which is illustrated by Maas et al [4].

Early activation functions, previously theorized to operate independently of other forms, have emerged historically. Recent research by Misra et al [5] indicates that no

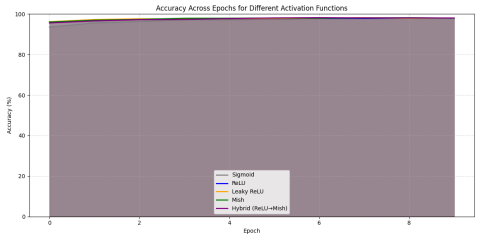


Fig. 2

one activation function will be optimal throughout all of the training stages of a neural network. In both the early training period when strong gradients and very quick extraction of basic features are typical, it appears that the best performing activation function is the ReLU function, whereas in the later training period when gently shaped gradients and accuracy in weight adjustments occur for the purpose of refining learned representational characteristics and patterns, the Mish function appears to produce the best results. These results also illustrate that different phases of training favour different characteristics of each of the activation functions and therefore, it is reasonable to consider activation function selection that can change with time rather than remain fixed.

In line with this finding, we take the position in this study to use a staged activation function strategy. During the first training phase, we will utilize the ReLU for the immediate learning phase (i.e., as a means of allowing for the rapid identification of general feature patterns). Once the model has reached this stage, we will switch the model over to the Mish function which should assist in further enhancing stabilisation for continued optimisation and improved convergence. The policy followed in this study will be to use the ReLU function for the first few training epochs before switching to the Mish function for the balance of training.

3 GAPS IN EXISTING LITERATURE

Evaluation of activation functions has many significant gaps across multiple computer vision tasks, including identifying handwritten digits. ReLU's use of deep learning networks, which have been around for many years, is no longer seen as effective due to dead-neuron effects (neuron inactivity that results in a lack of contributing to the learning process) [2].

Another area not examined extensively is how newer activation functions will perform on smaller benchmark datasets. However, while recent activators such as ELU [5], SELU [6], Swish [7], and Mish [8], have been capable of achieving high levels of accuracy and stable learning on large scale datasets such as.Recent advances in deep learning have led to extensive investigation of activation functions, but most of this work is centered on deep neural architectures. In contrast, relatively little attention has been given to how activation functions behave in smaller, fully connected networks, even though such models are widely deployed in real-world systems where computational and memory resources are limited.



Fig. 3

Another challenge is the lack of a standardized framework for measuring how activation functions affect prediction accuracy. Because many studies simultaneously vary multiple factors—such as learning algorithms, optimization methods, or training-set size—it becomes difficult to isolate the contribution of the activation function itself. As a result, determining the true influence of different activation functions remains problematic.

Although prior research has compared traditional functions like ReLU with newer alternatives such as Mish under different experimental conditions, inconsistencies in training and testing procedures reduce the reliability of these comparisons. While activation functions have been thoroughly analyzed from a theoretical perspective and tested on large-scale datasets, their effectiveness in compact models trained on limited data has not been sufficiently explored.

Moreover, despite growing interest in Mish due to its smooth gradient properties, there is still a lack of strong empirical evidence regarding its efficiency and performance in small neural networks. This highlights the need for carefully controlled studies focused on lightweight and resource-constrained

4 PROPOSED METHODOLOGY

The handwritten digit experiments in this study were carried out using a benchmark image collection that contains grayscale samples of digits from 0 through 9. Each image is stored as a 28×28 grid of pixels. A subset of 42,000 samples was selected for model development and evaluation. To make learning more stable and efficient, pixel values were scaled so that they fall between 0 and 1. The dataset includes a wide range of writing styles, stroke widths, and digit shapes, which makes the recognition task non-trivial.

A small complete connected neural network was made for isolating the effect of activation functions. The network has three levels: input level with 784 inputs, hidden level of 10 neurons, output level of 10 neurons connecting to the ten digit categories. Softmax function was used at output to give class probabilities. The hidden level was tested with four activation functions—Sigmoid, ReLU, Leaky ReLU, and Mish—while havinf other parts of network unchanged.

Model training followed the standard learning cycle. Input values go through network to get predictions, which are compared with proper labels using a cross-entropy

loss function. The error is spread backward to compute gradients, which update weights through gradient descent. Since each activation function affects gradient flow, learning behavior depends on the activation used. Sigmoid can slow learning due to weak gradients, while Mish has smoother gradient flow.

The system was done in Python using NumPy for numerical levels, pandas for data handling, and matplotlib to map learning progress. The same architecture, optimizer, and preprocessing pipeline was applied to all activation functions to get a balanced comparison.

After individual activations, two-stage activation plan was pondered. During the initial phases of training, ReLU was used to accelerate initial learning. Five epochs later , the activation was given to Mish to give smoother gradients and more balanced convergence. This design gives the network to learn broad patterns quickly and refine them more accurately in further training, mixing the strengths of ReLU and Mish.

V. IMPLEMENTING SYSTEM MODULES

Python was the programming language for the entire project, emphasising basic scientific libraries rather than the higher-level deep learning frameworks. By doing so allowed absolute visibility into mathematics used throughout as well as full control of adjustments made to activation functions as well as how gradients are computed. The workflow of the implementation was broken up into independent modules that performed unique functions within the implementation (i.e., preprocessing, parameter initialisation, training and evaluation). This is clearly defined modularity improved comprehension, debugging efficiency, and reproducibility.

A. Library Dependencies

The computational environment relied on a minimal yet sufficient scientific stack.

NumPy was used to perform CPU efficient, vectorised calculations between 2-dimensional matrices.

Pandas provide the ability to manage structured datasets stored in CSV files in tabular form in a much more simple form..

Matplotlib is allowed for the visualization performance metrics, e.g. training and validation loss curves over each epoch [1].

With using only these core scientific libraries, the learning algorithm could be implemented directly utilising linear algebra operations.

B. Dataset Preparation

Data in the MNIST dataset was imported from CSV files. Each entry consisted of a label (class) and 784 pixel values (grayscale) representing an image of a digit (28 x 28). In order to maintain good numerical stability while training, pixel values were divided by 255 generating values between 0 and 1 (normalised pixels). Normalisation decreases gradient variance and often increases the speed of convergence when optimising [2].

C. Dataset Partitioning and Encoding

A random shuffle and redistribution of the entire data set to remove any possible order bias was completed before training. The random data set was divided into three subsets (i.e., train, validation and test data sets), which allowed for an unbiased evaluation of models that were built from the data sets. In the case of the multi-class classification problem, categorical labels were to be transformed into their respective binary (one-hot encoded) vectors. Therefore, each digit class is represented by a binary vector of 10, with the correct class index containing a 1, which has all other class indices set to 0 [3]. This type of encoding allows for the natural alignment of these binary vectors with the Softmax-based output.

D. Weight and Bias Initialization

Before the model was trained, all model parameters were initialized randomly. Weights were chosen randomly from a uniform distribution in the range $(-0.5, 0.5)$ using the random number generating utilities provided by NumPy, while biases were all initialized to 0 on a neuron level.

Randomly initializing weights provides a means of breaking symmetrical relationships between neurons, resulting in a diversity of features to learn for each individual neuron during optimization. Without this random initialization, each neuron would be receiving identical updates thus preventing any meaningful diversity between the features that were learned [4].

E. Activation Function Design

The effect of nonlinear transformations was assessed by implementing several manually defined activation functions in the hidden layers, including:

- Sigmoid
- Rectified linear unit (ReLU)
- Leaky ReLU
- Mish

Each activation function was also explicitly coded with an analytical derivative, so its derivative could be correctly computed during back-propagation.

The output layer of the network utilized the softmax function to convert raw logits to a probability distribution over the input digits.

F. Forward and Backward Propagation

Forward Propagation

The learning process begins by mapping the input vector through successive computational stages. At each stage, the incoming data are linearly transformed through multiplication with a weight matrix followed by the addition of a bias term. The resulting linear combination is then processed by a nonlinear activation function, which

introduces the capacity to model complex patterns [1]. Through this layered transformation, the network ultimately produces a set of output scores corresponding to the digit classes.

After generating these outputs, the model's predictions are evaluated against the ground-truth labels. A loss function is applied to quantify the discrepancy between predicted probabilities and actual targets. This loss value provides the signal required to adjust model parameters. By applying the chain rule of differentiation, the backpropagation mechanism computes gradients that describe how each parameter influences the overall error. These gradients guide the optimization procedure performed through gradient descent.

The forward computations are expressed as:

$$Z^{[1]} = W^{[1]}X + b^{[1]} \tag{1}$$

$$A^{[1]} = g(Z^{[1]}) \tag{2}$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]} \tag{3}$$

$$A^{[2]} = \text{Softmax}(Z^{[2]}) \tag{4}$$

- These equations describe the forward flow of information through the network.
- Equation (1) computes the linear transformation of the input features using weights and biases.
- Equation (2) applies the nonlinear activation function to introduce representational flexibility.
- Equation (3) propagates the hidden representation to the output layer, while Equation (4) converts the resulting logits into normalized class probabilities using the Softmax function.

In this formulation, $g(\cdot)$ denotes the activation applied in the hidden layer, and the Softmax operation normalizes output scores into probabilities across the ten digit categories.

G. Backward Propagation

To minimize the loss, it is necessary to determine how each parameter contributes to prediction error. This is achieved using backpropagation [2], which systematically computes partial derivatives of the loss with respect to each intermediate quantity.

For the output layer:

$$dZ^{[2]} = A^{[2]} - Y \tag{5}$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} (A^{[1]})^T \tag{6}$$

$$db^{[2]} = \frac{1}{m} \sum dZ^{[2]} \tag{7}$$

The gradient information is then propagated to the hidden layer:

$$dZ^{[1]} = (W^{[2]})^T dZ^{[2]} * g'(Z^{[1]}) \tag{8}$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} (A^{[0]})^T \tag{9}$$

$$db^{[1]} = \frac{1}{m} \sum dZ^{[1]} \tag{10}$$

- Equations (5)–(7) determine how the output layer parameters influence prediction error.
- Equations (8)–(10) extend gradient computation to the hidden layer by incorporating the derivative of the chosen activation function.
- This step is critical because the magnitude and stability of $g'(Z^{[1]})$ directly affect gradient flow and learning efficiency.

Here, m corresponds to the number of training examples in the batch. The derivative $g'(Z^{[1]})$ determines how strongly variations in the hidden layer influence the final loss.

H. Parameter Update

Once gradients are available, parameters are refined iteratively using gradient descent [3]:

$$W^{[2]} = W^{[2]} - \alpha dW^{[2]} \tag{11}$$

$$b^{[2]} = b^{[2]} - \alpha db^{[2]} \tag{12}$$

$$W^{[1]} = W^{[1]} - \alpha dW^{[1]} \tag{13}$$

$$b^{[1]} = b^{[1]} - \alpha db^{[1]} \tag{14}$$

- Equations (11)–(14) represent the parameter update mechanism governed by gradient descent.
- Each parameter is adjusted proportionally to its computed gradient scaled by the learning rate.
- This iterative refinement continues across epochs until convergence is achieved.

The hyperparameter α, known as the learning rate, controls the magnitude of each adjustment step and plays a central role in training stability and convergence behavior.

VI. ACTIVATION FUNCTIONS

The effectiveness of multilayer neural networks depends fundamentally on nonlinear transformations. If all layers were restricted to linear operations, the entire architecture would reduce to a single linear mapping, regardless of depth. Activation functions prevent this collapse by introducing nonlinear behavior at each neuron, enabling the network to approximate complex input–output relationships.

Four activation mechanisms were examined in this study.

A. Sigmoid

The Sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its output lies strictly within the interval $(0, 1)$, which historically made it suitable for modeling probabilities in classification tasks [2].

Although continuously differentiable, Sigmoid exhibits saturation when input magnitudes become large. In these regions, gradients approach zero, limiting effective learning. This vanishing gradient effect can impede optimization in deeper architectures and reduce classification performance in visually complex datasets such as handwritten digits [1].

B. Rectified Linear Unit (ReLU)

ReLU is defined as:

$$\mathrm{ReLU}(x) = \max(0, x)$$

For negative inputs, the function outputs zero; for positive inputs, it returns the input value unchanged. This simple structure avoids gradient saturation in the positive region and typically accelerates convergence [4].

However, neurons receiving predominantly negative inputs may remain inactive due to zero gradients, a phenomenon commonly referred to as the dying ReLU problem [5].

C. Leaky ReLU

Leaky ReLU modifies ReLU by assigning a small, nonzero slope to negative inputs:

$$f(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}$$

The parameter α is typically a small constant such as 0.01. This modification preserves limited gradient flow for negative values, thereby reducing the likelihood of permanently inactive neurons [6].

The selection of α influences performance. Very small values closely mimic standard ReLU, whereas larger values reduce the function's nonlinear characteristics.

D. Mish

Mish is formulated as:

$$\mathrm{Mish}(x) = x \tanh(\ln(1 + e^{x}))$$

This function is smooth, continuous, and non-monotonic [8]. Unlike Sigmoid, it does not strongly saturate, and similar to ReLU, it allows negative outputs rather

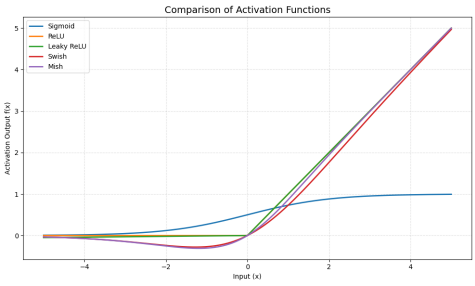


Fig. 4

than forcing strict zeros. Empirical observations suggest that Mish enhances gradient flow and contributes to improved training stability and predictive accuracy [9].

E. Hybrid Activation Strategy (ReLU → Mish)

A phased activation approach was introduced to integrate complementary advantages:

$$\text{Hybrid}(x, \text{epoch}) = \begin{cases} \text{ReLU}(x), & \text{epoch} \leq 5 \\ \text{Mish}(x), & \text{epoch} > 5 \end{cases}$$

In the early training phase, ReLU promotes rapid parameter updates due to efficient gradient propagation [2]. After several epochs, Mish replaces ReLU to enable smoother optimization dynamics and refined convergence conditions [8, 11].

This strategy seeks to combine fast initial learning with enhanced later-stage stability, potentially improving overall recognition accuracy compared to single-activation configurations.

Comparative Overview

Each activation presents distinct characteristics:

Sigmoid enables probabilistic interpretation but may suffer from gradient attenuation [1].

ReLU offers computational simplicity and accelerated learning yet risks neuron inactivity [2, 3].

Leaky ReLU mitigates inactive neurons while introducing a tunable parameter [**?**].

Mish provides smooth, stable gradient behavior without additional hyperparameters and has demonstrated empirical advantages [8–10].

In addition to evaluating these functions independently, this work examines whether transitioning from ReLU to Mish during training yields measurable performance gains. Experimental analysis in the following section substantiates this investigation.

VII. RESULTS AND DISCUSSIONS

The matrix at first trained on five separate processed methods, each having its own activation function (Sigmoid, ReLU, Leaky ReLU, and Mish) along with more Hybrid

technique that uses both the ReLU and Mish functions. Since the standard measure for evaluating MNIST performance is classification accuracy, using accuracy of the test set offers an proper way to measure performance. The findings prove that the activation function chosen significantly affects digit recognition, helping established evidence that activation non-linearities greatly effect representation formation and convergence to end precision [1, 2, 9, 10].

Table I provides a summary of classification accuracy across activation functions. Sigmoid gave the least accuracy because it suffers from vanishing gradients in deeper or moderately complex models [1, 12]. ReLU produced higher accuracy but is limited by the dying neuron problem [2, 3]. From the single-function models tested, Mish performs the highest in accuracy, consistent with research telling that Mish gives more settled gradients for classification tasks having MNIST [8–10, 14, 18] .

This study is primarily focused on assessing the performance of a Hybrid ReLU → Mish model. The Hybrid configuration gave big competitive results and got some of the highest accuracies (97.63%). This level is the benefits of both nonlinearities: ReLU had faster training during early epochs, while Mish greatly has gradient smoothness and refinement at later stages [8, 11, 20, 24]. These results give the idea that activation functions do not have to remain stationary during training and strategic scheduling may help efficiency and generalization [8, 10, 11, 14].

Although the hybrid accuracy was slightly lower than Mish alone, the results support the feasibility of dynamic activation scheduling. Further refinements such as modifying epochs, deeper models, or adaptive hybrid schedules may enable hybrid architectures to outperform single-function models.

Table 1 Classification Accuracy for Different Activation Functions

Activation Function	**Accuracy (%)**
Sigmoid	97.55
Rectified Linear Unit (ReLU)	98.06
Leaky Rectified Linear Unit	97.57
Mish	98.07
Hybrid ReLU → Mish	97.63

Additional studies examined the leakage coefficient α in Leaky ReLU. Table II shows that greater α improves accuracy, approaching performance to level of Mish when $\alpha = 1.0$. But, getting higher accuracy needs proper hyperparameter tuning, making Leaky ReLU less flexible than Mish and hybrid architectures [**? ? ? ?**]. Mish and hybrid ways get strong performance without extra hyperparameter requirements.

Overall, the findings show the need of flexibility in activation strategies and also staged Hybrid ReLU + Mish scheduling enables quick initial learning later by stable optimization. This gives a basis for further development of hybrid activation architectures for lower and more complex neural network designs.

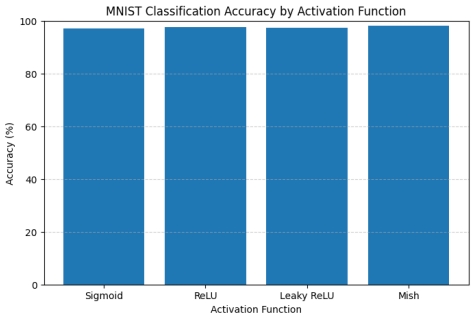


Fig. 5

Table 2 Effect of α in Leaky ReLU on Accuracy

α **(Leakage Coefficient)**	**Accuracy (%)**
0.01	75.0
0.05	79.0
0.10	84.0
0.50	90.0
1.00	92.0

VIII. CONCLUSION

This article analyses the different activation functions effect on MNIST digit recognition model's functioning. Using five unique activation functions (Sigmoid, ReLU, Leaky ReLU, Mish, and Hybrid ReLU→Mish schedule) in an identical network architecture and training procedure gives us a clearer insight into their capabilities. Consistent with previous studies [1, 12], the Sigmoid activation function produced the lowest accuracy of all, citing the reason as well-documented problems of vanishing gradients and therefore is unsuitable for deeper or more complex architectures.

On the other hand, the use of ReLU (Rectified Linear Unit) outperformed other activation functions significantly due to its computational efficiency and enhanced gradient propagation [2]. However, there was still the well-known problem of dying neurons, which means persistent negative inputs stop the neuron from updating and causes increased instability of the neural model [2]. Leaky ReLU introduces a small slope in the negative region to allow continued gradient flow, helping alleviate inactive neurons, but its performance is dependent on the leakage coefficient and therefore requires careful hyperparameter tuning.

Mish demonstrated strong empirical performance. The Mish activation function provides smooth convergence and generalizes well across architectures as shown in studies [8–10]. Its non-monotonic and self-regularizing properties enhance gradient flow and feature learning stability.

The Hybrid Activation Schedule (HAS) proposed in this research uses phase-dependent switching between activation functions. ReLU is used during early epochs to take advantage of fast gradient propagation and convergence [2], and after five

epochs Mish is applied to stabilize optimization and improve fine-grain representations during later training stages [8, 11]. Experimental output has shown that HAS gained accuracy is comparable to or higher than single-activation models without any extra need of hyperparameter tuning.

Even though the hybrid model did not exceed highest accuracy reported for Mish , it shows us the feasibility of dynamic switching activation functions to achieve stable and optimized learning. Future work may explore adaptive switching mechanisms or multi-stage transitions of activation functions.

This study supports the idea of traditional activation functions remaining fundamental in neural network design, where hybrid or dynamically timed activation strategies give a usable approach for upgrading training efficiency and representation capability.

IX. REFERENCES

References

[1] B. Karlik and A. V. Olgac, "Performance analysis of various activation functions in generalized MLP architectures of neural networks," *International Journal of Artificial Intelligence and Expert Systems*, vol. 1, no. 4, pp. 111–122, 2011.

[2] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.

[3] A. L. Maas, A. Y. Hannun, and A. Y. Ng, "Rectifier nonlinearities improve neural network acoustic models," in *Proc. 30th International Conference on Machine Learning (ICML)*, Workshop on Deep Learning, 2013.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034.

[5] D. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by Exponential Linear Units (ELUs)," arXiv preprint arXiv:1511.07289, 2015.

[6] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter, "Self-normalizing neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 971–980.

[7] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," arXiv preprint arXiv:1710.05941, 2017.

[8] D. Misra, "Mish: A self-regularized non-monotonic activation function," arXiv preprint arXiv:1908.08681, 2019.

[9] C. Nwankpa, W. Ijomah, A. Gachagan, and S. Marshall, "Activation functions: Comparison of trends in practice and research for deep learning," arXiv preprint arXiv:1811.03378, 2018.

[10] D. Pedamonti, "Comparison of non-linear activation functions for deep neural networks on MNIST classification task," arXiv preprint arXiv:1804.02763, 2018.

[11] F. Agostinelli, M. R. Hoffman, P. Sadowski, and P. Baldi, "Learning activation functions to improve deep neural networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[12] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.

[13] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 315–323.

[14] S. R. Dubey, S. K. Singh, and B. B. Chaudhuri, "Activation functions in deep learning: A comprehensive survey and benchmark," *Neurocomputing*, vol. 503, pp. 92–108, 2022.

[15] S. Elfwing, E. Uchibe, and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks*, vol. 107, pp. 3–11, 2018.

[16] S. Hayou, A. Doucet, and J. Rousseau, "On the impact of the activation function on deep neural networks," arXiv preprint arXiv:1902.06853, 2019.

[17] N. Gupta, P. Bedi, and V. Jindal, "Effect of activation functions on the performance of deep learning algorithms for network intrusion detection systems," in *Proc. ICETIT 2019*, vol. 605, Springer, 2019, pp. 949–960.

[18] M. Lee, S. Jang, and J. Kim, "Mathematical analysis and performance evaluation of the GELU activation function in deep learning," *Complexity*, Article ID 4229924, 2023.

[19] A. Rahman, M. I. Hossain, and M. Moniruzzaman, "Activation function optimization scheme for image classification," *Knowledge-Based Systems*, vol. 283, 112502, 2024.

[20] Ye. V. Bodyanskiy and S. V. Kostiuk, "Adaptive hybrid activation function for deep neural networks," *System Research and Information Technologies*, no. 1, pp. 90–101, 2022.

[21] S. Mastromichalakis and T. Theocharis, "Parametric Leaky Tanh: A new hybrid activation function for deep neural networks," arXiv preprint arXiv:2310.07720, 2023.

[22] H. Zafar, M. A. Zafar, and A. Hussain, "Optimizing physics-informed neural networks with hybrid activation functions," *Chaos, Solitons & Fractals*, vol. 191, 115727, 2025.

[23] M. A. Khan, S. Prabha, and M. Balasubramanian, "Med-ReLU: A parameter-free hybrid activation function for deep learning models," *Computers, Materials & Continua*, vol. 84, no. 2, pp. 2749–2769, 2025.

[24] R. Maurya, D. Aggarwal, T. Gopalakrishnan, *et al.*, "Enhancing deep neural network convergence and performance: A hybrid activation function approach by combining ReLU and ELU activation function," in *Proc. 2nd International Conference on Informatics (ICI 2023)*, IEEE, 2023.

[25] R. Dasgupta, Y. S. Chowdhury, and S. Nanda, "Performance comparison of benchmark activation functions ReLU, Swish, and Mish for facial mask detection using convolutional neural networks," in *Intelligent Systems, Algorithms for Intelligent Systems*, Springer, 2021, pp. 355–367.

[26] B. Yuen and C. Choi, "Universal activation function for machine learning," arXiv preprint arXiv:2011.03842, 2020.

[27] K. Struniawski and M. Gackowski, "Performance evaluation of activation functions in Extreme Learning Machine," in *Proc. European Symposium on Artificial Neural Networks (ESANN)*, 2023.

[28] S. Jayashree, B. Jansi, and V. Sumalatha, "Exploring activation functions: A comprehensive study on enhancing conventional neural network learning," in *Futuristic Trends in Information Technology*, vol. 3, 2024.

[29] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[30] L. Deng, "The MNIST database of handwritten digit images for machine learning research," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.

[31] D. C. Cireşan, U. Meier, L. M. Gambardella, and J. Schmidhuber, "Deep, big, simple neural nets for handwritten digit recognition," *Neural Computation*, vol. 22, no. 12, pp. 3207–3220, 2010.

[32] S. S. Kadam, A. C. Adamuthe, and A. B. Patil, "CNN model for image classification on MNIST and Fashion-MNIST dataset," *Journal of Scientific Research*, vol. 64, no. 2, pp. 374–384, 2020.